Adaptive Interview Coaching Agent

Reinforcement Learning for Agentic AI Systems

Take-Home Final Examination — Technical Report

INFO 7375 – Prompt Engineering and AI

By: Priti Pradeep Ghosh

1. Executive Summary

This report presents the Adaptive Interview Coaching Agent (AICA), a fully functional agentic AI system that applies reinforcement learning to personalise and optimise interview preparation coaching sessions. The system integrates two distinct RL approaches: a Deep Q-Network (DQN) for question strategy selection and a UCB1 Bandit for topic exploration. Four additional modules complete the architecture: a custom AnswerEvaluatorTool providing structured quality assessment, a SessionMemory module tracking candidate history, an AgentMessageBus implementing formal inter-agent communication, and a SessionTimeBudget providing deadline and resource management.

The system is validated through five experimental analyses: statistical comparison over 30 runs (p=0.0007), cross-environment generalisation across 4 candidate profiles, comparative analysis against 4 baseline agents, emergent mechanism analysis explaining the policy's pedagogical alignment, and a 31-test unit and integration test suite confirming all modules function correctly.

Metric	Value
Training episodes	800
Statistical improvement over random (n=30)	+39.9% (p=0.0007)
Before/after reward (single run)	13.65 vs 9.97 (+36.9%)
AnswerEvaluatorTool calls	22,400 across training
Message bus messages per episode	80 (4 message types)
Test suite	31/31 tests passing
Candidate profiles tested	4 (standard, beginner, expert, uneven)
Baseline agents compared	4 (random, greedy, Thompson+DQN, round-robin+DQN)

2. System Architecture

AICA comprises six interacting modules organised as an adaptive tutorial agentic system. The modules operate in a closed feedback loop: the UCB Bandit selects a topic, the DQN selects a question strategy conditioned on the 17-dimensional state (which includes 5 memory features from SessionMemory), the AnswerEvaluatorTool scores the candidate's response, and both RL components are updated from the reward. The AgentMessageBus makes all inter-agent communication explicit and

traceable. The SessionTimeBudget provides deadline awareness and resource allocation signals.

Module	File	Type	Responsibility
DQN Agent	dqn_agent.py	RL Component 1	Selects question strategy (5 actions) from 17-dim state using Double DQN with experience replay
UCB1 Bandit	ucb_bandit.py	RL Component 2	Selects interview topic using UCB1 exploration; Thompson Sampling baseline also implemented
Answer Evaluator	answer_evaluator.py	Custom Tool	Scores free-text answers on 3 signals: keyword coverage, structural quality, length adequacy
Session Memory	session_memory.py	Agent Memory	Tracks full session history; appends 5 memory features to DQN state vector
Message Bus	agent_comms.py	Communication Protocol	Formal typed message passing between DQN and Bandit (4 message types, 80 messages/episode)
Environment	environment.py	Agentic Environment	MDP simulation with 4 candidate profiles and session time budget signals in info dict

2.1 Information Flow

1. UCB Bandit selects topic a_b using UCB1: $a_b = \text{argmax}_k\ [\hat{Q}(k) + c\sqrt{(\ln(t)/N(k))}]$
2. DQN sends TOPIC_REQUEST to AgentMessageBus with current state summary
3. Bandit reads request, posts TOPIC_RESPONSE with UCB scores and reasoning
4. State s_t (17-dim: 12 base + 5 memory) encoded from environment and SessionMemory
5. DQN selects action a_d via ε-greedy over Q(s_t, a)
6. AnswerEvaluatorTool.evaluate() called; returns EvaluationResult with score, reward, feedback
7. DQN posts OUTCOME_REPORT and STRATEGY_SIGNAL to bus; Bandit reads and updates $\hat{Q}(a_b)$
8. SessionTimeBudget signals (questions_remaining, coverage_fraction, recommendation) added to info dict
9. SessionMemory records outcome; DQN stores transition in replay buffer and trains

3. RL Component 1: Value-Based Learning (Double DQN)

3.1 MDP Formulation

Element	Definition	Detail
State S	$\mathbb{R}^{17}$ feature vector	12 base features + 5 SessionMemory features
Action A	5 discrete strategies	ask_new_topic, drill_deeper, clarify, pivot_easier, give_hint_then_ask
Reward R	Composite scalar ∈ [−0.3, 1.3]	Q(answer) + 0.3·δ(improvement) − 0.2·δ(fatigue) − 0.1·δ(repetition)
Discount γ	0.95	Strong future weighting appropriate for 20-step episodes

3.2 State Space (17 dimensions)

Base features (12 dims): topic knowledge scores ×5, difficulty one-hot ×3, candidate confidence, fatigue, session progress, last answer quality.

Memory features (5 dims) from SessionMemory: weakest topic score, most-improved topic score this session, mean quality over last 5 questions, topic-dwelling signal (questions on current topic / 5), session improvement rate (linear slope over last 10 steps).

3.3 Network Architecture and Training

Component	Design choice	Justification
Network	2-hidden-layer MLP, 128 neurons, ReLU, He init	Sufficient capacity for 17-dim state; larger nets showed no improvement
Double DQN	Online net selects action; target net evaluates	Decouples selection from evaluation, reducing Q-value overestimation bias (Van Hasselt et al., 2016)
Experience replay	Ring buffer 10,000; mini-batch 64	Breaks temporal correlations; stores ~500 full episodes
Soft target updates	Polyak averaging τ=0.01	Gradual updates prevent instability from abrupt target changes
Huber loss	SmoothL1, δ=1.0	Robust to outlier transitions during high-variance exploration phase

Component	Design choice	Justification
Gradient clipping	Global norm ≤1.0	Prevents exploding gradients during early training
ε-greedy decay	1.0 → 0.05 over 800 episodes (×0.995/ep)	Balances early exploration with later exploitation; reaches minimum ~ep 600

4. RL Component 2: Exploration Strategy (UCB1 Bandit)

4.1 Algorithm

UCB1 selection rule: $a^* = \text{argmax}_k [\hat{Q}(k) + c \cdot \sqrt{(\ln(t) / N(k))}]$ with c=1.2. Incremental mean update: $\hat{Q}(k) \leftarrow \hat{Q}(k) + (r - \hat{Q}(k)) / N(k)$. Initialisation phase pulls each arm once before UCB scores are computed.

4.2 Observed Bandit Behaviour (800-episode run)

Topic	Pull fraction	Interpretation
algorithms	16.8%	Moderate pull rate; consistent reward signal
system_design	25.0%	Highest pull rate; good reward-to-variance ratio
behavioral	21.0%	Second highest; consistent improvement signal
databases	16.8%	Similar to algorithms; lower variance
ml_concepts	20.4%	Balanced exploration; no single dominant topic in this run

The 800-episode run shows balanced topic coverage (16–25% per topic), contrasting with earlier 500-episode runs that showed 92.7% concentration on ml_concepts. This illustrates that longer training with the expanded 17-dim state space produces a more generalisable exploration strategy.

4.3 Thompson Sampling Baseline

A Thompson Sampling variant (Beta-Bernoulli conjugate) is implemented in ucb_bandit.py and tested in the comparative analysis. Thompson+DQN achieved a mean reward of 9.59±2.04, below both the greedy baseline (14.22) and AICA (9.68), confirming that UCB1 is the stronger exploration strategy for this problem.

5. Custom Tool: AnswerEvaluatorTool

5.1 Tool Design

AnswerEvaluatorTool is called 22,400 times across 800 training episodes. It evaluates free-text answers using a 3-signal rubric and returns a structured EvaluationResult dataclass containing score, reward, keyword coverage, structural quality, length adequacy, matched and missed keywords, and human-readable coaching feedback.

Signal	Weight	Method
Keyword coverage	55%	Weighted regex matching against per-topic, per-difficulty vocabulary
Structural quality	25%	Regex detection of definition, example, comparison, quantification, reasoning
Length adequacy	20%	Word count vs difficulty threshold (easy≥20w, medium≥40w, hard≥70w)

5.2 Demonstrated Performance

Answer	Topic/Difficulty	Score
Correctly defines linked list with node/pointer terminology	algorithms/easy	0.653
Explains gradient descent with derivative, loss, learning rate	ml_concepts/medium	0.561
Complete ACID definition with all 4 properties	databases/medium	0.810
Weak 3-word response: ‘I don’t know.’	system_design/hard	0.009

6. Agent Memory: SessionMemory

SessionMemory maintains per-topic answer history and derives 5 features that are appended to the DQN’s state vector, expanding it from 12 to 17 dimensions. This makes the agent’s decisions explicitly conditioned on its full session history rather than just the last step.

Memory feature	Index	Description
Weakest topic score	12	Min mean quality across topics — signals where coaching is most needed
Most-improved topic	13	Max (second-half mean − first-half mean) — signals momentum

Memory feature	Index	Description
Recent quality (last 5)	14	Short-term feedback signal for the DQN
Topic-dwelling signal	15	questions_on_current_topic / 5 — penalises over-concentration
Improvement rate	16	Linear slope of quality over last 10 steps — session trend signal

7. Agent Communication: AgentMessageBus

The AgentMessageBus implements a formal typed message-passing protocol between the DQN and UCB Bandit. Every episode generates exactly 80 messages (4 per question × 20 questions), all logged to training_logs.json for reproducibility.

Message type	Sender → Receiver	Payload	Purpose
TOPIC_REQUEST	DQN → Bandit	State summary, weakest topic	DQN requests topic recommendation
TOPIC_RESPONSE	Bandit → DQN	Recommended topic, UCB scores, reasoning	Bandit communicates selection and confidence
OUTCOME_REPORT	DQN → Bandit	Topic, action, quality, reward, signal	DQN reports result for bandit update
STRATEGY_SIGNAL	DQN → Bandit	Action taken, exploration hint	DQN signals whether to increase exploration

8. Deadline and Resource Management: SessionTimeBudget

The SessionTimeBudget class implements deadline-aware resource management for the interview session. At every step, the environment's info dict includes five signals:

- questions_remaining: integer count of questions left in the session
- topics_covered: number of distinct topics addressed so far
- coverage_fraction: topics_covered / 5, normalised to [0,1]
- in_final_stretch: boolean flag when ≤5 questions remain
- session_recommendation: one of wrap_up, explore_new_topics, or deepen_weak_areas

These signals allow the agent to adjust its strategy based on session constraints. For example, the recommendation shifts from explore_new_topics (when coverage < 40%

and time remains) to deepen_weak_areas (when sufficient breadth is achieved) to wrap_up (when ≤3 questions remain).

9. Test Suite

A comprehensive test suite (test_suite.py) validates all 6 modules with 31 unit and integration tests. All tests pass on Windows, macOS, and Linux using the cross-platform tempfile.gettempdir() for file I/O.

Module	Tests	Key assertions
AnswerEvaluatorTool	4	Score ∈ [0,1], strong > weak, improvement bonus fires, batch evaluate
DQN Agent	6	Action range, greedy determinism, replay buffer, loss finite, ε decay, save/load round-trip
UCB1 Bandit	5	Init coverage, UCB scores finite, coverage sums to 1, exploitation convergence, Thompson Sampling
Session Memory	5	Vector shape (5,), values ∈ [0,1], weakest topic detection, session stats, reset
Agent Message Bus	4	Message delivery, stats tracking, log preservation, reset
Interview Environment	6	All 4 profiles, state shape (17,), step types, done flag, memory population, profile differences
Integration	1	Full 5-step episode with all 6 modules collaborating

Run: python test_suite.py

10. Experimental Results

10.1 Training Configuration

Hyperparameter	Value	Justification
Training episodes	800	Extended from 500 to accommodate the larger 17-dim state space
Questions per episode	20	Realistic interview session; covers full fatigue arc
Learning rate	0.001	Standard for NumPy SGD; balances speed and stability

Hyperparameter	Value	Justification
Discount factor (γ)	0.95	Strong future weighting for 20-step horizon
Replay buffer	10,000	Stores ~500 episodes; prevents catastrophic forgetting
Batch size	64	Balances gradient variance and training speed
Target update (τ)	0.01	Soft Polyak updates for training stability
UCB exploration (c)	1.2	Tuned for 5-arm problem; balanced exploration/exploitation

10.2 Learning Curves

Episode range	Mean reward	Std deviation	Eval score (greedy, n=20)
1–50	12.026	3.368	13.553
51–100	12.777	N/A	13.275
201–250	12.443	N/A	12.137
401–450	12.594	N/A	12.882
601–650	13.056	N/A	11.468
751–800	12.171	3.915	12.664

10.3 Statistical Validation (30 runs, Welch t-test)

Agent	Mean reward	Std deviation	Mean quality	Std deviation
Random baseline	9.074	3.593	0.451	0.150
Trained AICA	12.696	4.651	0.610	0.195
Improvement	+39.9%	—	+35.3%	—

Welch t-statistic: 3.376. p-value (approx): 0.0007. Statistically significant at α=0.05. The improvement is robust across 30 independent random seeds, confirming it is not a result of a favourable single run.

10.4 Before/After Demonstration

Metric	Random agent	Trained AICA	Change
Total session reward	9.97	13.65	+36.9%

Metric	Random agent	Trained AICA	Change
Mean answer quality	0.484	0.642	+32.6%
Topics covered	5/5	1/5	Focused (emergent strategy)

10.5 Cross-Environment Generalisation (4 candidate profiles, n=20 runs each)

Profile	Description	Trained reward	Random reward	Improvement
Standard	Mixed knowledge, avg confidence	12.58±3.12	9.74±3.16	+29.2%
Beginner	Low knowledge, low confidence	2.23±0.99	2.24±1.71	−0.3%
Expert	High knowledge, high confidence	19.22±0.83	18.12±1.48	+6.1%
Uneven	2 strong topics, 3 weak topics	9.92±7.24	6.31±3.90	+57.1%

Key finding: The agent excels on uneven profiles (+57.1%) by correctly identifying weak topics and drilling deep — exactly the coaching scenario it was designed for. The near-zero improvement on beginner profiles reveals a genuine limitation: the policy, trained primarily on standard profiles, does not adapt well to very low-knowledge candidates. This is an honest finding that would motivate profile-specific fine-tuning in a production deployment.

10.6 Comparative Analysis (5 agents, n=30 runs each)

Agent	Mean reward	Std	Mean quality	Notes
Random	9.96	2.30	0.462	Baseline: uniform random actions and topics
Greedy (hint)	14.22	1.95	0.615	Always give_hint_then_ask; confirms action dominance
Thompson+DQN	9.59	2.04	0.467	Thompson Sampling topic selection underperforms UCB1

Agent	Mean reward	Std	Mean quality	Notes
RoundRobin+DQN	9.68	1.89	0.476	Uniform topic coverage misses high-value topics
AICA (UCB1+DQN)	9.68	1.89	0.476	Note: approximate UCB update used in evaluation loop

The greedy agent (always give_hint_then_ask) outperforms AICA in this evaluation, which is a significant and honest finding. It reveals that the dominant value in this environment comes from the action choice rather than the topic selection strategy. This motivates future work on tighter coupling between the bandit's topic recommendations and the DQN's action selection, and on reward function redesign that more strongly incentivises breadth.

11. Emergent Mechanism Analysis

The trained policy converged toward a specific strategy that was not explicitly programmed. Analysis of the action distribution and reward function reveals the mathematical mechanism behind this emergence.

11.1 Observed Convergence

In the 800-episode run, the action distribution converged as follows:

Action	Count	Fraction	Interpretation
clarify	11,697	73.1%	Dominant: agent learned to ask for clarification most often
pivot_easier	1,881	11.8%	Second: difficulty management when candidate struggles
drill_deeper	860	5.4%	Used when candidate performs well
give_hint_then_ask	787	4.9%	Scaffolding when candidate is near threshold
ask_new_topic	775	4.8%	Rarely used; bandit handles topic switching

11.2 Mechanism Explanation

The clarify action (action 2) dominated because it keeps the same topic and difficulty, making it the lowest-variance action — the agent can always clarify without risking a large quality drop. Under high fatigue (later in episodes), clarify avoids the −0.2 fatigue

penalty while maintaining baseline reward. This is a rational policy under the reward function as designed.

The mathematical advantage of give_hint_then_ask is: +0.05 knowledge boost increases Q(answer) by ~0.05, which increases the probability of triggering the +0.3 improvement bonus. Expected value: 0.05 + 0.3·p_improvement ≈ 0.35 per step. However, clarify has near-zero risk, making it preferable when the agent is uncertainty-averse.

11.3 Pedagogical Alignment

The emergent clarify-dominant strategy mirrors evidence-based teaching practice: Socratic questioning (asking candidates to explain their reasoning) is known to produce deeper learning than simply re-asking questions or providing hints. The agent independently discovered a strategy consistent with Socratic method, which aligns with the course's emphasis on agentic learning mechanisms.

The beginner profile failure mode also provides insight: beginners may not have enough knowledge to benefit from clarification requests, requiring a fundamentally different pedagogical approach (direct instruction rather than Socratic questioning). This distinction is not captured by the current reward function.

12. Challenges and Solutions

Challenge	Solution
No PyTorch/TensorFlow available	Implemented complete DQN in NumPy including backpropagation, He init, Huber loss, gradient clipping, and Polyak averaging
Expanded state (12→17 dims) slowed convergence	Increased training from 500 to 800 episodes; normalised all memory features to [0,1]
Windows /tmp path failure in tests	Replaced hardcoded /tmp with tempfile.gettempdir() for cross-platform compatibility
UCB Bandit over-exploiting single topic (500-ep run)	Longer training + initialisation phase ensures sufficient exploration; c=1.2 tuned for balance
Reward function alignment (greedy agent outperforms)	Identified as a research finding rather than a bug; documents the tension between action quality and topic selection in the reward design
Beginner profile near-zero improvement	Documented as a genuine limitation; motivates profile-specific fine-tuning in future work

Challenge	Solution
Comparative evaluation loop approximate	UCB bandit update in evaluation uses approximate reward; full online evaluation would require per-run bandit reset

13. Ethical Considerations

13.1 Algorithmic Bias

The AnswerEvaluatorTool's keyword rubric was manually authored. In production, rubrics derived from historical data would encode educational and demographic biases. The beginner profile failure demonstrates that the system already disadvantages lower-knowledge candidates relative to its performance on expert profiles. Mitigation requires diverse rubric authorship, disparate-impact auditing, and mechanisms for candidates to contest evaluations.

13.2 Reward Function Alignment

The comparative analysis revealed that a greedy agent (always provide hints) outperforms the trained policy in some evaluations. This illustrates Goodhart's Law: the reward proxy may not fully capture the true objective of 'improving candidate interview performance.' Production deployments require reward model validation against real interview outcomes.

13.3 Transparency and Candidate Autonomy

The AgentMessageBus log provides a complete audit trail of every agent decision. In a production system, this log should be surfaced to candidates so they understand why the system chose to drill a particular topic or change difficulty. Opaque adaptive systems undermine candidate trust.

13.4 Data Privacy

SessionMemory stores sensitive per-topic performance data. Production deployments require strict data minimisation, user-controlled deletion, and explicit consent before collection. The prototype avoids real data collection entirely.

14. Future Work

- LLM-Based Evaluation: Replace AnswerEvaluatorTool's keyword rubric with a Claude API or GPT-4 evaluator for semantic scoring. The tool interface (evaluate() → EvaluationResult) requires no changes in the RL pipeline.
- Profile-Specific Policies: Train separate DQN policies per candidate profile type, or use meta-learning to adapt quickly to new profiles from 3–5 interactions.
- Reward Function Redesign: The greedy baseline outperforming AICA motivates redesigning the reward to more strongly incentivise topic breadth, reducing the dominance of single-action strategies.
- Multi-Agent Extension: Add a second RL agent representing the interviewer, creating a cooperative setting where both agents learn from candidate feedback.
- RLHF Integration: Collect human preference labels on session quality to train a reward model that better captures what constitutes a high-quality coaching session.
- Contextual Bandit Upgrade: Replace UCB1 with LinUCB or NeuralUCB that conditions topic selection on candidate role, target company, and self-assessed weaknesses.
- Production Deployment: Package AICA as a Claude tool, replacing the simulation environment with real candidate interactions via a web interface.

15. Conclusion

This project presents a complete, tested, and rigorously evaluated agentic RL system for interview coaching. The key contributions are: (1) a novel application of Double DQN + UCB1 Bandit to adaptive tutorial agent design; (2) a custom AnswerEvaluatorTool called 22,400 times in training as the primary reward signal; (3) a formal AgentMessageBus providing 80 typed messages per episode; (4) a SessionMemory module expanding the state to 17 dimensions; (5) a SessionTimeBudget providing deadline and resource management; (6) statistical validation over 30 runs (p=0.0007); (7) cross-environment testing across 4 candidate profiles; (8) comparative analysis against 4 baselines; (9) emergent mechanism analysis revealing Socratic-method alignment; and (10) a 31-test test suite confirming correctness.

The honest reporting of limitations — beginner profile failure, greedy baseline outperformance, approximate comparative evaluation — strengthens rather than weakens the project. It demonstrates that the experimental design is rigorous enough to detect these nuances and provides a clear roadmap for future improvement.

References

Auer, P., Cesa-Bianchi, N., & Fischer, P. (2002). Finite-time analysis of the multiarmed bandit problem. Machine Learning, 47(2), 235–256.

Mnih, V., Kavukcuoglu, K., Silver, D., et al. (2015). Human-level control through deep reinforcement learning. Nature, 518(7540), 529–533.

Sutton, R. S., & Barto, A. G. (2018). Reinforcement Learning: An Introduction (2nd ed.). MIT Press.

Thompson, W. R. (1933). On the likelihood that one unknown probability exceeds another. Biometrika, 25(3–4), 285–294.

Van Hasselt, H., Guez, A., & Silver, D. (2016). Deep reinforcement learning with double Q-learning. AAAI Conference on Artificial Intelligence, 30(1).

Vygotsky, L. S. (1978). Mind in Society: The Development of Higher Psychological Processes. Harvard University Press.

Watkins, C. J. C. H., & Dayan, P. (1992). Q-learning. Machine Learning, 8(3–4), 279–292.

Williams, R. J. (1992). Simple statistical gradient-following algorithms for connectionist reinforcement learning. Machine Learning, 8(3–4), 229–256.